Von Null bis Live: Der komplette Guide für deine Restaurant-SaaS

> *Dieser Guide erklärt dir alles, was du brauchst, um deine Restaurant-Management-App vom Code auf deinem Laptop bis zur fertigen, öffentlich erreichbaren Anwendung zu bringen. Geschrieben für absolute Anfänger — keine Vorkenntnisse nötig.*

Inhaltsverzeichnis

1. Das Datenkonstrukt
2. GitHub
3. Der Tech Stack
4. Server & Hosting
5. Coolify
6. Vom Code zum Live-Produkt
7. Sicherheit
8. Launch
9. Kostenzusammenfassung
10. ASCII-Gesamtübersicht

---

1. DAS GESAMTE DATENKONSTRUKT

1.1 Was sind Datenbanken? (Erklärt wie für einen 5-Jährigen)

Die Restaurant-Analogie

Stell dir vor, du hast ein riesiges **Aktenschrank-System** in deinem Restaurant-Büro:

- **Die Datenbank** = Der gesamte Aktenschrank
- **Eine Tabelle** = Eine Schublade im Schrank (z.B. "Schublade: Reservierungen")
- **Ein Feld (Spalte)** = Die Beschriftung auf jedem Karteikarten-Feld (z.B. "Name", "Datum", "Uhrzeit")
- **Eine Zeile (Datensatz)** = Eine einzelne ausgefüllte Karteikarte

Beispiel:

```
Schublade "Reservierungen" (= Tabelle)
┌──────────────┬──────────┬──────────┬──────────┐
│ Gastname     │ Telefon  │ Datum    │ Personen │  ← Felder (Spalten)
├──────────────┼──────────┼──────────┼──────────┤
│ Müller       │ 0151-... │ 05.04.26 │ 4        │  ← 1. Datensatz (Zeile)
│ Schmidt      │ 0170-... │ 06.04.26 │ 2        │  ← 2. Datensatz (Zeile)
│ Weber        │ 0162-... │ 06.04.26 │ 6        │  ← 3. Datensatz (Zeile)
└──────────────┴──────────┴──────────┴──────────┘
```

Warum nicht einfach eine Excel-Tabelle?

Excel funktioniert für 1 Person, die allein arbeitet. Aber sobald:

- 10 Kellner gleichzeitig Bestellungen aufgeben
- 50 Restaurants gleichzeitig die App nutzen
- Daten sicher und dauerhaft gespeichert werden müssen

...brauchst du eine echte Datenbank. Sie kann Tausende gleichzeitige Anfragen verarbeiten, hält die Daten konsistent (keine Konflikte) und ist viel schneller.

1.2 Welche Datenbank-Typen gibt es?

Es gibt zwei große Familien:

Relationale Datenbanken (SQL)

Daten sind in **Tabellen mit festen Spalten** organisiert — wie Excel-Tabellen, die miteinander verknüpft sind.

Datenbank	Beschreibung	Preis
PostgreSQL	Mächtigste Open-Source-DB. Industriestandard.	Kostenlos
MySQL	Weit verbreitet, etwas simpler als PostgreSQL.	Kostenlos
SQLite	Mini-DB in einer einzigen Datei. Für kleine Apps.	Kostenlos

Nicht-Relationale Datenbanken (NoSQL)

Daten sind in **flexiblen Dokumenten** gespeichert — eher wie JSON-Dateien als wie Tabellen.

Datenbank	Beschreibung	Preis
MongoDB	Beliebt für schnelle Prototypen.	Kostenlos
Redis	Extrem schnell, für Zwischenspeicher (Cache).	Kostenlos

Welche ist die richtige für eine Multi-Tenant SaaS?

PostgreSQL. Eindeutig.

Warum:

1. **Beziehungen zwischen Daten** — Eine Bestellung gehört zu einem Tisch, der zu einem Restaurant gehört. Das bildet PostgreSQL perfekt ab.
2. **Datenkonsistenz** — Wenn eine Bestellung gespeichert wird, garantiert PostgreSQL, dass der Tisch auch wirklich existiert. MongoDB macht das nicht automatisch.
3. **Multi-Tenant-Sicherheit** — Mit PostgreSQL kannst du pro Abfrage sicherstellen, dass Restaurant A nur seine eigenen Daten sieht. Nie die von Restaurant B.
4. **DSGVO-Compliance** — Relationale Datenbanken machen es einfacher, personenbezogene Daten gezielt zu löschen (z.B. "lösche alle Reservierungen älter als 30 Tage").
5. **Kostenlos und bewährt** — Wird von Instagram, Spotify und Stripe genutzt.

1.3 Das Datenbankschema deiner Restaurant-SaaS

Ein **Schema** ist der Bauplan deiner Datenbank — welche Tabellen es gibt, welche Felder jede Tabelle hat, und wie sie zusammenhängen.

Übersicht aller Tabellen

```
┌──────────────────┐
│   restaurants    │  ← Der "Hauptmieter" (Tenant)
│  (Stammdaten)    │
└──────────┬───────┘
           │ hat viele...
     ┌─────┼──────────────────────────────────┐
     │     │                                  │
     ▼     ▼          ▼          ▼            ▼
┌────────┐ ┌──────────┐ ┌──────────┐ ┌────────────┐ ┌──────────┐
│ tische │ │kategorien│ │mitarbeiter│ │reservierung│ │ schichten│
└────────┘ └──────────┘ └──────────┘ └────────────┘ └──────────┘
     │           │
     │           ▼
     │       ┌─────────┐
     │       │ gerichte │
     │       └─────────┘
     │
     ▼
┌──────────────┐
│ bestellungen │
└──────────┬───┘
           │ hat viele...
           ▼
┌──────────────────┐
│bestellpositionen │
└──────────────────┘
```

Was jede Tabelle speichert

restaurants — Die Stammdaten jedes Restaurants (= Tenant)

```
- id               → Eindeutige ID (UUID, z.B. "a3f2e1d0-...")
- name             → "Pizzeria Roma"
- logo_url         → Link zum Logo-Bild
- oeffnungszeiten  → "Mo-Fr 11-22, Sa-So 12-23"
- strasse, plz,    → Adresse
  stadt
- telefon, email   → Kontaktdaten
- waehrung         → "EUR"
- primaerfarbe     → "#ea580c" (Orange — die Farbe der Gästeseite)
- lizenz_code      → Einmaliger Aktivierungscode
- max_mitarbeiter  → Wie viele Mitarbeiter die Lizenz erlaubt
- abo_status       → "trial" / "active" / "expired"
```

kategorien — Speisekarten-Kategorien (Vorspeisen, Hauptgerichte, Getränke...)

```
- id
- restaurant_id    → Gehört zu welchem Restaurant
- name             → "Hauptgerichte"
- reihenfolge      → Sortierung (0, 1, 2...)
```

tische — Die physischen Tische im Restaurant

```
- id
- restaurant_id
- nummer           → Tischnummer (1, 2, 3...)
- kapazitaet       → Maximale Personenzahl
- status           → "frei" / "besetzt" / "wartet_auf_zahlung"
- qr_url           → Link für den QR-Code am Tisch
```

gerichte — Die einzelnen Gerichte auf der Speisekarte

```
- id
- restaurant_id
- kategorie_id     → Gehört zu welcher Kategorie
- name             → "Margherita"
- beschreibung     → "Tomate, Mozzarella, Basilikum"
- preis            → 9.50
- bild_url         → Link zum Foto
- allergene        → "Gluten, Laktose"
- verfuegbar       → true/false (Ausverkauft-Schalter)
```

bestellungen — Eine Bestellung pro Tisch

```
- id
- restaurant_id
- tisch_id          → Welcher Tisch hat bestellt
- status            → "offen" / "in_zubereitung" / "serviert" / "bezahlt"
- gesamtpreis       → Automatisch berechnet
- anmerkung         → "Ohne Zwiebeln bitte"
```

bestellpositionen — Die einzelnen Gerichte in einer Bestellung

```
- id
- bestellung_id     → Gehört zu welcher Bestellung
- gericht_id        → Welches Gericht
- menge             → Wie oft bestellt (z.B. 2x Margherita)
- einzelpreis       → Preis zum Zeitpunkt der Bestellung (wichtig!)
```

> ***Warum "einzelpreis" extra speichern?*** *Weil der Preis eines Gerichts sich ändern kann. Wenn die Margherita nächste Woche 10€ kostet statt 9.50€, soll die alte Bestellung trotzdem den richtigen Preis zeigen.*

reservierungen — Tischreservierungen

```
- id
- restaurant_id
- tisch_id          → Optional: konkreter Tisch zugewiesen
- gast_name         → "Familie Müller"
- telefon           → ⚠️ DSGVO: wird nach 30 Tagen gelöscht
- datum             → Wann die Reservierung ist
- personen          → Wie viele Personen kommen
- status            → "ausstehend" / "bestaetigt" / "storniert"
- quelle            → "app" / "whatsapp" / "telefon"
```

mitarbeiter — Die Benutzerkonten

```
- id
- restaurant_id
- name              → "Max Mustermann"
- email             → ⚠️ DSGVO: personenbezogen
- passwort_hash     → Verschlüsseltes Passwort (NIE Klartext!)
- rolle             → "admin" / "kellner" / "kueche"
- aktiv             → Konto aktiv oder gesperrt
```

schichten — Dienstplan

```
- id
- restaurant_id
- mitarbeiter_id   → Welcher Mitarbeiter
- datum            → Welcher Tag
- beginn, ende     → Von wann bis wann
- notiz            → "Bitte Terrasse aufbauen"
```

Beziehungen zwischen den Tabellen

Tabellen sind über sogenannte **Foreign Keys** (Fremdschlüssel) verbunden. Das ist wie ein Verweis: "Diese Bestellung gehört zu Tisch Nr. 5".

```
restaurants ──1:N──→ tische
restaurants ──1:N──→ kategorien
restaurants ──1:N──→ mitarbeiter
restaurants ──1:N──→ reservierungen
restaurants ──1:N──→ bestellungen
restaurants ──1:N──→ schichten
kategorien  ──1:N──→ gerichte
tische      ──1:N──→ bestellungen
bestellungen──1:N──→ bestellpositionen
gerichte    ──1:N──→ bestellpositionen
mitarbeiter ──1:N──→ schichten
```

1:N bedeutet: "Eins zu Viele". Ein Restaurant hat VIELE Tische. Ein Tisch gehört zu genau EINEM Restaurant.

1.4 Was bedeutet Multi-Tenancy?

Tenant = Mieter. Jedes Restaurant ist ein "Mieter" in deiner App.

Die Wohnhaus-Analogie

Stell dir ein Mehrfamilienhaus vor:

- Das **Haus** = Deine App
- Jede **Wohnung** = Ein Restaurant
- Der **Vermieter** = Du (der SaaS-Anbieter)

Jede Wohnung hat eigene Räume, eigene Möbel, eigene Schlüssel. Familie Müller kann nicht in die Wohnung von Familie Schmidt rein.

Genauso funktioniert Multi-Tenancy:

- Alle Restaurants teilen sich EINE Datenbank (= ein Haus)
- Aber jedes Restaurant sieht NUR seine eigenen Daten (= eigene Wohnung)
- Das wird über die `restaurant_id` sichergestellt

Wie das technisch funktioniert

Jede Datenbankabfrage enthält IMMER einen Filter:

```
-- ❌ FALSCH: Zeigt ALLE Tische aller Restaurants
SELECT * FROM tische;

-- ✅ RICHTIG: Zeigt nur die Tische von Restaurant "abc-123"
SELECT * FROM tische WHERE restaurant_id = 'abc-123';
```

In deiner App passiert das automatisch — die `restaurant_id` wird aus dem Login-Token des Mitarbeiters gelesen. Ein Kellner von Pizzeria Roma kann niemals die Bestellungen von Sushi Palace sehen.

Die drei Ansätze für Multi-Tenancy

Ansatz	Beschreibung	Für wen?
Shared Database, Shared Schema	Alle Daten in denselben Tabellen, getrennt durch `restaurant_id`	**Dein Ansatz!** Am einfachsten, skaliert gut
Shared Database, Separate Schemas	Jeder Tenant hat eigene Tabellen innerhalb einer DB	Mittelgroße Apps
Separate Databases	Jeder Tenant hat seine eigene Datenbank	Enterprise (teuer)

Du nutzt Ansatz 1 — der ist für den Start perfekt.

1.5 Wie Daten durch die App fließen

Hier der komplette Weg einer Bestellung, von der Eingabe bis zur Anzeige:

```
  GAST                    FRONTEND                  BACKEND
DATENBANK
   |                        |                         |                       |
   |  1. Klickt "2x         |                         |                       |
   |     Margherita         |                         |                       |
   |     bestellen"         |                         |                       |
   | ———————————————————→   |                         |                       |
   |                        |  2. Baut JSON-Objekt:   |                       |
   |                        |  {                      |                       |
   |                        |    tisch_id: "...",     |                       |
   |                        |    positionen: [{       |                       |
   |                        |      gericht_id: "...", |                       |
   |                        |      menge: 2           |                       |
   |                        |    }]                   |                       |
   |                        |  }                      |                       |
   |                        |                         |                       |
   |                        |  3. Sendet HTTP POST    |                       |
   |                        |     an /api/bestellungen |                      |
   |                        | ———————————————————→    |                       |
   |                        |                         |  4. Prüft die Daten:  |
   |                        |                         |     - Tisch existiert? |
   |                        |                         |     - Gericht verfügbar?
   |                        |                         |     - Preis aus DB holen
   |                        |                         |     (NIE vom Client!) |
   |                        |                         |                       |
   |                        |                         |  5. INSERT INTO       |
   |                        |                         |     bestellungen ...  |
   |                        |                         | ———————————————————→|
   |                        |                         |                       |
   |                        |                         |  6. INSERT INTO       |
   |                        |                         |     bestellpositionen |
   |                        |                         | ———————————————————→|
   |                        |                         |                       |
   |                        |                         |  ←—— 7. Bestätigung  |
   |                        |                         |       "Gespeichert"   |
   |                        |                         |                       |
   |                        |  ←—— 8. JSON-Antwort:   |                       |
   |                        |       { id: "...",      |                       |
   |                        |         status: "offen"} |                      |
   |                        |                         |                       |
   |  ←—— 9. "Bestellung    |                         |                       |
   |        aufgegeben!"    |                         |                       |
   |        (Grüner Haken)  |                         |                       |
```

Gleichzeitig in der Küche:

```
  KÜCHE-TABLET              FRONTEND                  BACKEND
DATENBANK
    |                        |                         |                       |
    |                        |  10. WebSocket-         |                       |
    |                        |      Verbindung hört    |                       |
    |                        |      auf neue           |                       |
    |                        |      Bestellungen       |                       |
    |                        |                         |                       |
    |                        |  ←── 11. "Neue          |                       |
    |                        |       Bestellung für    |                       |
    |                        |       Tisch 5!"         |                       |
    |                        |       (Socket.io Push)  |                       |
    |                        |                         |                       |
    |  ←── 12. Bestellung    |                         |                       |
    |       erscheint auf    |                         |                       |
    |       dem Bildschirm   |                         |                       |
    |                        |                         |                       |
    |  13. Koch klickt       |                         |                       |
    |      "In Zubereitung"  |                         |                       |
    | ──────────────────────→ |                         |                       |
    |                        |  14. PUT                |                       |
    |                        |  /api/bestellungen/     |                       |
    |                        |  xyz/status             |                       |
    |                        | ──────────────────────→ |                       |
    |                        |                         |  15. UPDATE           |
    |                        |                         |  bestellungen SET     |
    |                        |                         |  status =             |
    |                        |                         |  'in_zubereitung'     |
    |                        |                         | ──────────────────────→|
```

In einfachen Worten:

1. **Der Gast** tippt auf dem Handy (oder Tablet am Tisch) seine Bestellung ein
2. **Das Frontend** (React-App) verpackt die Bestellung als JSON und schickt sie ans Backend
3. **Das Backend** (Node.js-Server) prüft alles, holt die echten Preise aus der DB
4. **Die Datenbank** (PostgreSQL) speichert die Bestellung dauerhaft
5. **Socket.io** schickt sofort eine Benachrichtigung an die Küche ("Neue Bestellung!")
6. **Die Küche** sieht die Bestellung live auf ihrem Bildschirm

2. GITHUB — WAS ES IST UND WIE ES FUNKTIONIERT

2.1 Git vs. GitHub — Der Unterschied

Die Analogie

- **Git** = Ein System, das jede Änderung an deinem Code aufzeichnet. Wie ein "Strg+Z" (Rückgängig) für dein gesamtes Projekt — aber unbegrenzt, und du kannst jeden früheren Zustand wiederherstellen.
- **GitHub** = Eine Website, auf der du deinen Git-Verlauf online speicherst. Wie Google Drive, aber speziell für Code.

Git ist das Werkzeug (läuft auf deinem Laptop). **GitHub** ist der Online-Speicher (die Cloud-Kopie).

Warum brauchst du das?

1. **Sicherheit** — Laptop kaputt? Kein Problem, dein Code ist auf GitHub.
2. **Verlauf** — Du siehst jede Änderung, die jemals gemacht wurde.
3. **Deployment** — Coolify (dein Hosting) holt den Code direkt von GitHub.
4. **Zusammenarbeit** — Falls du mal einen Entwickler einstellst, kann er direkt mitarbeiten.

2.2 Was ist ein Repository (Repo)?

Ein **Repository** ist ein Projekt-Ordner, der von Git überwacht wird.

Dein Repo `restaurant-saas` enthält:

- Den gesamten Code (Frontend + Backend)
- Die Datenbankdefinitionen
- Die Projektdokumentation
- Die KOMPLETTE Änderungshistorie (jeder Commit)

Wichtig: Das Repo speichert nicht nur die aktuellen Dateien, sondern den gesamten Verlauf. Du kannst zu jedem Zeitpunkt in der Geschichte zurückreisen.

2.3 Der komplette Git-Workflow

Schritt 1: Repository erstellen

Lokal (auf deinem Mac):

```
# In den Projektordner gehen
cd ~/restaurant-saas

# Git initialisieren (einmalig)
git init

# Ergebnis: Ein versteckter .git Ordner wird erstellt.
# Dieser Ordner enthält den gesamten Verlauf.
```

Auf GitHub:

1. Gehe auf github.com → Klick "New Repository"
2. Name: `restaurant-saas`
3. Sichtbarkeit: **Private** (wichtig! Dein Code ist geschäftlich)
4. KEIN README oder .gitignore anhaken (du hast das schon lokal)
5. "Create Repository" klicken

Lokal mit GitHub verbinden:

```
# GitHub als "Remote" hinzufügen (= Online-Ziel)
git remote add origin https://github.com/DEIN-USERNAME/restaurant-saas.git

# Hauptbranch benennen
git branch -M main
```

Schritt 2: Commit — Was bedeutet das?

Ein **Commit** ist ein Schnappschuss deines Projekts zu einem bestimmten Zeitpunkt.

Analogie: Stell dir vor, du fotografierst deinen Schreibtisch jeden Tag. Wenn du letzte Woche etwas auf dem Schreibtisch hattest und es jetzt nicht mehr findest, schaust du einfach das

Foto von letzter Woche an.

So machst du einen Commit:

```
# 1. Prüfe, was sich geändert hat
git status
# Zeigt dir z.B.:
#   modified: src/App.tsx         (geändert)
#   new file: src/pages/Login.tsx (neu erstellt)
#   deleted:  src/old-file.ts     (gelöscht)

# 2. Änderungen "staging" — auswählen was in den Commit soll
git add src/App.tsx src/pages/Login.tsx
# Oder ALLE Änderungen auf einmal:
git add .

# 3. Commit erstellen mit einer Nachricht die erklärt WAS du geändert hast
git commit -m "Login-Seite hinzugefügt"
```

Gute Commit-Nachrichten:

- `"Login-Seite hinzugefügt"` — klar und spezifisch
- `"Bestellungs-Status kann jetzt geändert werden"` — sagt was neu ist
- `"Bug behoben: Reservierung wurde doppelt gespeichert"` — sagt was repariert wurde

Schlechte Commit-Nachrichten:

- `"Update"` — sagt nichts aus
- `"fix"` — was wurde gefixt?
- `"asdf"` — nein.

Schritt 3: Push — Wo landen die Daten?

Push = Deine lokalen Commits zu GitHub hochladen.

```
git push origin main
# "Schiebe meine Commits zum Online-Speicher (origin), Branch main"
```

Danach ist dein Code auf zwei Orten:

1. Auf deinem Laptop (lokales Repo)
2. Auf GitHub (Remote Repo)

Schritt 4: Pull und Clone

Pull = Änderungen von GitHub auf deinen Laptop herunterladen.

```
git pull origin main
# "Hole die neuesten Änderungen vom Online-Speicher"
```

Clone = Ein Repo von GitHub zum ersten Mal auf deinen Laptop kopieren.

```
git clone https://github.com/DEIN-USERNAME/restaurant-saas.git
# Erstellt einen Ordner "restaurant-saas" mit dem gesamten Code + Verlauf
```

Wann brauchst du was?

- `clone` → Einmalig, wenn du das Projekt auf einem neuen Computer brauchst
- `pull` → Regelmäßig, um Änderungen zu holen (z.B. wenn Claude Code etwas geändert hat)
- `push` → Jedes Mal wenn du Änderungen hochladen willst

Schritt 5: Branches

Ein **Branch** (Zweig) ist wie eine parallele Kopie deines Projekts.

Analogie: Du arbeitest an einem wichtigen Brief. Bevor du eine riskante Änderung machst, kopierst du den Brief. Wenn die Änderung schlecht war, hast du noch das Original. Wenn sie gut war, ersetzt du das Original.

```
main ──────●──────────●──────────── (stabile Version)
           \          /
            ●──●──●              (feature/qr-codes)
            Neue QR-Code Funktion
```

```
# Neuen Branch erstellen und dorthin wechseln
git checkout -b feature/qr-codes

# Hier arbeitest du an der QR-Code Funktion...
# Wenn fertig: zurück zu main und zusammenführen
git checkout main
git merge feature/qr-codes
```

Für den Anfang brauchst du keine Branches. Arbeite direkt auf `main`. Branches werden wichtig, wenn du mehrere Features gleichzeitig entwickelst oder mit anderen zusammenarbeitest.

Schritt 6: .gitignore — Was NICHT hochgeladen wird

Die Datei `.gitignore` listet alles auf, was Git IGNORIEREN soll:

```
# Installierte Pakete (werden auf dem Server neu installiert)
node_modules/

# Umgebungsvariablen (GEHEIM!)
.env
.env.local

# Build-Dateien (werden auf dem Server neu gebaut)
dist/
build/

# Betriebssystem-Dateien
.DS_Store
Thumbs.db

# IDE-Dateien
.vscode/
.idea/
```

Was du NIEMALS hochladen darfst

Datei	Warum nicht?
`.env`	Enthält Passwörter, API-Keys, Datenbank-Zugänge
`*.pem`, `*.key`	SSH-Schlüssel und SSL-Zertifikate
`node_modules/`	Riesig (100+ MB), wird automatisch installiert
Datenbank-Dumps	Können echte Kundendaten enthalten

Wenn du aus Versehen ein Passwort gepusht hast: Das Passwort sofort ändern! Selbst wenn du den Commit löschst, ist er in der Git-Historie noch sichtbar.

2.4 Schritt-für-Schritt: Erstes Repo + erster Push

```
# 1. Terminal öffnen, in den Projektordner navigieren
cd ~/restaurant-saas

# 2. Git initialisieren (falls noch nicht geschehen)
git init

# 3. Alle Dateien zum Commit vorbereiten
git add .

# 4. Ersten Commit erstellen
git commit -m "Erster Commit: Restaurant SaaS Projekt"

# 5. GitHub-Repo als Remote hinzufügen
git remote add origin https://github.com/DEIN-USERNAME/restaurant-saas.git

# 6. Hochladen!
git push -u origin main

# Fertig! Dein Code ist jetzt auf GitHub.
```

> ***Hinweis:*** *Bei deinem Projekt ist das bereits erledigt — du hast schon 2 Commits auf GitHub.*

2.5 Was ist eine README.md?

Die **README.md** ist die "Startseite" deines Repos auf GitHub. Sie wird automatisch angezeigt, wenn jemand dein Repo öffnet.

Eine gute README enthält:

1. **Projektname und kurze Beschreibung** — Was macht die App?
2. **Tech Stack** — Welche Technologien werden verwendet?
3. **Installation** — Wie startet man das Projekt lokal?
4. **Umgebungsvariablen** — Welche `.env`-Werte werden benötigt?
5. **Lizenz** — Unter welchen Bedingungen darf der Code genutzt werden?

`.md` steht für **Markdown** — ein einfaches Format für Text mit Überschriften, Listen und Code-Blöcken. Diese Datei, die du gerade liest, ist auch Markdown.

---

3. DER KOMPLETTE TECH STACK

3.1 Übersicht — Was macht was?

```
┌──────────────────────────────────────────────────────────────┐
│                        DEIN TECH STACK                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Frontend (was der User sieht)                               │
│  ├── React          → Baut die Benutzeroberfläche            │
│  ├── TypeScript     → JavaScript mit Fehlerschutz            │
│  ├── Tailwind CSS   → Macht alles hübsch (Design)            │
│  └── Vite           → Startet den Entwicklungsserver          |
│                                                              │
│  Backend (was im Hintergrund läuft)                          │
│  ├── Node.js        → Führt JavaScript auf dem Server aus     |
│  ├── Express        → Verarbeitet HTTP-Anfragen (API)         |
│  ├── TypeScript     → JavaScript mit Fehlerschutz            │
│  └── Socket.io      → Live-Updates (Küche sieht neue Bestellung)|
│                                                              │
│  Datenbank                                                   │
│  └── PostgreSQL     → Speichert alle Daten dauerhaft          |
│                                                              │
│  Authentifizierung                                           │
│  ├── JWT            → Login-Token ("Ausweis" des Users)       |
│  └── bcrypt         → Verschlüsselt Passwörter                |
│                                                              │
│  Hosting & Deployment                                        │
│  ├── Hetzner VPS    → Der Server auf dem alles läuft          |
│  ├── Coolify        → Verwaltungstool für den Server          |
│  └── GitHub         → Code-Speicher + automatisches Deployment |
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

3.2 Was jede Schicht macht und warum

Frontend — React + TypeScript + Tailwind + Vite

Was es macht: Alles, was der Benutzer sieht und anfasst — Buttons, Formulare, Tabellen, Menüs.

Analogie: Das Frontend ist wie die Einrichtung deines Restaurants — die Speisekarte, die Dekoration, die Tische. Der Gast interagiert nur damit.

Technologie	Was sie macht	Warum sie?
React	Baut die Benutzeroberfläche aus Bausteinen ("Komponenten")	Industriestandard. Riesige Community = viele Hilfe-Ressourcen
TypeScript	JavaScript, aber mit Typ-Prüfung. Fängt Fehler VOR dem Ausführen ab	Verhindert Bugs wie "undefined is not a function"
Tailwind CSS	CSS-Framework mit fertigen Klassen (`bg-blue-500` , `p-4` , `rounded-lg`)	Schneller als eigenes CSS schreiben, konsistentes Design
Vite	Entwicklungsserver. Zeigt Code-Änderungen sofort im Browser	Extrem schnell (< 1 Sekunde)

Backend — Node.js + Express + TypeScript

Was es macht: Verarbeitet alle Anfragen, prüft Berechtigungen, spricht mit der Datenbank.

Analogie: Das Backend ist wie die Küche deines Restaurants. Der Gast sieht sie nicht, aber hier passiert die eigentliche Arbeit.

Technologie	Was sie macht	Warum sie?
Node.js	Führt JavaScript außerhalb des Browsers aus (auf dem Server)	Gleiche Sprache wie im Frontend = weniger Lernaufwand
Express	Mini-Framework für HTTP-Server. Definiert die API-Routen	Simpel, bewährt, leichtgewichtig
TypeScript	Gleiche Vorteile wie im Frontend	Konsistenz im gesamten Projekt
Socket.io	Echtzeit-Kommunikation (Server → Browser, ohne dass der Browser fragt)	Küche sieht neue Bestellungen sofort, ohne Seite neu zu laden

Datenbank — PostgreSQL

Was es macht: Speichert alle Daten dauerhaft und zuverlässig.

Analogie: Das Archiv deines Restaurants. Hier liegen alle Bestellungen, alle Reservierungen, alle Rezepte — sicher aufbewahrt.

Authentifizierung — JWT + bcrypt

JWT (JSON Web Token): Nach dem Login bekommt der User einen "Ausweis" (Token). Bei jeder weiteren Anfrage zeigt er diesen Ausweis vor. Das Backend prüft: "Ist der Ausweis

gültig? Was darf diese Person?"

bcrypt: Verschlüsselt Passwörter so, dass niemand sie lesen kann — nicht mal du. Wenn jemand die Datenbank stiehlt, sieht er nur unlesbaren Zeichensalat.

3.3 Wie die Teile miteinander kommunizieren

Was ist eine API?

API = Application Programming Interface = Schnittstelle zwischen Frontend und Backend.

Analogie: Die API ist wie der Kellner in deinem Restaurant. Der Gast (Frontend) sagt dem Kellner (API) was er will. Der Kellner bringt die Bestellung in die Küche (Backend). Die Küche bereitet es zu (Datenbank-Abfrage) und der Kellner bringt das Ergebnis zurück.

Was ist REST?

REST ist ein Regelwerk für APIs. Die wichtigsten Regeln:

HTTP-Methode	Bedeutung	Beispiel
GET	Daten abrufen	`GET /api/tische` → Alle Tische holen
POST	Neue Daten erstellen	`POST /api/bestellungen` → Neue Bestellung
PUT/PATCH	Daten aktualisieren	`PATCH /api/tische/5/status` → Tisch-Status ändern
DELETE	Daten löschen	`DELETE /api/tische/5` → Tisch löschen

Ein kompletter API-Aufruf

```
Frontend (Browser)                                Backend (Server)
       |                                                  |
       |  GET /api/speisekarte                            |
       |  Headers: {                                      |
       |    Authorization: "Bearer eyJhbGci..."           |
       |  }                                               |
       | ————————————————————————————————————————————→ |
       |                                                  |
       |                    Antwort (JSON):               |
       |  {                                               |
       |    "gerichte": [                                 |
       |      {                                           |
       |        "id": "abc-123",                          |
       |        "name": "Margherita",                     |
       |        "preis": 9.50,                            |
       |        "kategorie": "Pizza"                      |
       |      },                                          |
       |      ...                                         |
       |    ]                                             |
       |  }                                               |
       | ←———————————————————————————————————————————— |
```

JSON (JavaScript Object Notation) ist das Format, in dem Daten zwischen Frontend und Backend ausgetauscht werden. Es sieht aus wie strukturierter Text mit geschweiften Klammern und Anführungszeichen.

3.4 Umgebungsvariablen (.env Dateien)

Was sind Umgebungsvariablen?

Analogie: Stell dir vor, dein Rezept sagt "Füge die geheime Zutat hinzu". Die geheime Zutat steht nicht im Rezept (= Code), sondern auf einem Zettel im Tresor (= .env Datei).

Umgebungsvariablen sind **Einstellungen, die sich je nach Umgebung ändern** und **geheim bleiben müssen**.

Beispiel `.env` Datei

```
# Datenbank-Verbindung
DATABASE_URL=postgresql://user:password@localhost:5432/restaurant_db

# JWT-Geheimnis (zum Verschlüsseln der Login-Tokens)
JWT_SECRET=ein-sehr-langes-zufälliges-geheimnis-hier

# Server-Port
PORT=3001

# Frontend-URL (für CORS-Einstellungen)
FRONTEND_URL=http://localhost:5173
```

Warum nicht direkt im Code?

```
// ❌ NIEMALS so:
const db = connect("postgresql://admin:MeinPasswort123@server/db")

// ✅ IMMER so:
const db = connect(process.env.DATABASE_URL)
```

Gründe:

1. **Sicherheit** — Wenn der Code auf GitHub liegt, kann jeder das Passwort lesen
2. **Flexibilität** — Lokal nutzt du `localhost`, auf dem Server eine andere Adresse
3. **Trennung** — Verschiedene Umgebungen (Entwicklung, Test, Produktion) brauchen verschiedene Werte

Die verschiedenen Umgebungen

Umgebung	`.env` Wert für DATABASE_URL	Beschreibung
Lokal (Entwicklung)	`postgresql://localhost:5432/restaurant_dev`	Dein Laptop
Produktion (Live)	`postgresql://db-server:5432/restaurant_prod`	Der echte Server

4. SERVER & HOSTING — VON NULL ERKLÄRT

4.1 Was ist ein Server?

Analogie: Ein Server ist ein Computer, der rund um die Uhr läuft und auf Anfragen wartet. Wie ein Anrufbeantworter, der nie schläft.

Wenn jemand deine Website aufruft (`www.deine-app.de`), fragt sein Browser deinen Server: "Hey, gib mir die Seite." Und der Server antwortet mit dem HTML, CSS, JavaScript und den Daten.

Wo steht der Server physisch? In einem **Rechenzentrum** — ein Gebäude voller Computer in Regalen (Racks), mit redundanter Stromversorgung, Kühlung und Internetanbindung. Bei Hetzner steht das in Falkenstein (Sachsen) oder Nürnberg.

4.2 Server-Typen im Vergleich

Typ	Analogie	Beschreibung	Preis/Monat
Shared Hosting	WG-Zimmer	Du teilst einen Server mit Hunderten anderen. Langsam, eingeschränkt.	3-10€
VPS	Eigene Wohnung	Ein physischer Server wird in virtuelle Server aufgeteilt. Du hast deinen eigenen Bereich.	4-20€
Dedicated Server	Eigenes Haus	Ein ganzer physischer Server nur für dich.	50-200€+
Cloud (AWS, GCP)	Hotel	Du mietest genau die Ressourcen die du brauchst, zahlst pro Minute. Kann teuer werden.	Variable

Empfehlung für dein Projekt

Hetzner VPS — Typ CX22 (oder vergleichbar)

Eigenschaft	Wert	Was das bedeutet
RAM	4 GB	Arbeitsspeicher — wie viel die App gleichzeitig "im Kopf behalten" kann. 4 GB reicht für App + Datenbank + Coolify.
CPU	2 vCPU	Prozessorkerne — wie viele Aufgaben gleichzeitig verarbeitet werden. 2 reicht für den Start.
SSD	40 GB	Festplatte — wo alles gespeichert wird. 40 GB reicht für Code + Datenbank für lange Zeit.
Bandbreite	20 TB/Monat	Wie viel Datenverkehr möglich ist. 20 TB ist MASSIV — das wirst du nie ausschöpfen.
Standort	Falkenstein (DE)	Server steht in Deutschland — wichtig für DSGVO!
Preis	~4,50€/Monat	

Warum Hetzner?

- Deutsche Firma → Server in Deutschland → DSGVO-konform
- Extrem günstig für die Leistung
- Gute Dokumentation auf Deutsch
- Perfekt für den DACH-Markt

4.3 Was ist eine IP-Adresse?

Analogie: Jedes Haus hat eine Adresse (Straße + Hausnummer). Jeder Server hat eine IP-Adresse (z.B. `168.119.85.42`).

Wenn du deinen Hetzner-Server bestellst, bekommst du eine IP-Adresse. Damit erreichst du deinen Server von überall auf der Welt.

```
Du tippst: 168.119.85.42    →  Dein Browser verbindet sich mit dem Server
Du tippst: deine-app.de     →  DNS übersetzt das in die IP  →  gleicher Server
```

4.4 Was ist SSH?

SSH (Secure Shell) = Eine verschlüsselte Verbindung zu deinem Server. Damit kannst du Befehle auf dem Server ausführen, als würdest du direkt davor sitzen.

Analogie: SSH ist wie ein sicherer Telefonkanal zu deinem Server. Du gibst Befehle, der Server führt sie aus.

```
# So verbindest du dich mit deinem Server:
ssh root@168.119.85.42

# "root" = Der Haupt-Administrator-Account
# "168.119.85.42" = Die IP-Adresse deines Servers

# Nach der Eingabe wirst du nach dem Passwort gefragt.
# Danach bist du "auf dem Server" und kannst Befehle ausführen.
```

Sicherer: SSH-Schlüssel statt Passwort

Statt ein Passwort einzugeben, kannst du einen SSH-Schlüssel nutzen. Das ist wie ein spezieller digitaler Schlüssel, der auf deinem Laptop liegt. Nur wer den Schlüssel hat, kommt rein.

```
# SSH-Schlüssel erstellen (einmalig, auf deinem Mac):
ssh-keygen -t ed25519

# Schlüssel auf den Server kopieren:
ssh-copy-id root@168.119.85.42

# Ab jetzt: Kein Passwort mehr nötig!
ssh root@168.119.85.42  # → geht direkt rein
```

4.5 Domain, DNS und wie alles zusammenhängt

Was ist eine Domain?

Domain = Der Name deiner Website, z.B. `restaurant-manager.de`

Niemand merkt sich `168.119.85.42`. Deshalb kaufst du einen lesbaren Namen, der auf die IP-Adresse zeigt.

Was ist DNS?

DNS (Domain Name System) = Das "Telefonbuch des Internets".

Es übersetzt: `restaurant-manager.de` → `168.119.85.42`

Wie du eine Domain kaufst und verbindest

Schritt 1: Domain kaufen

- Anbieter: **Hetzner** (direkt beim Server-Anbieter) oder **Namecheap**
- Preis: ca. 10-15€/Jahr für eine `.de`-Domain
- Einfach den gewünschten Namen suchen, kaufen, fertig.

Schritt 2: DNS-Einträge setzen

In der DNS-Verwaltung deines Domain-Anbieters erstellst du einen **A-Record**:

```
Typ     Name    Wert            TTL
A       @       168.119.85.42   3600
A       www     168.119.85.42   3600
```

Was das bedeutet:

- **A-Record** = "Address Record" — Verbindet einen Namen mit einer IP-Adresse
- **@** = Die Domain selbst (`restaurant-manager.de`)
- **www** = Die www-Variante (`www.restaurant-manager.de`)
- **168.119.85.42** = Die IP deines Servers
- **TTL 3600** = "Time to Live" — Wie lange (in Sekunden) die Info zwischengespeichert wird

Nach 5-30 Minuten ist die Verbindung aktiv. Dann erreichst du deinen Server unter `restaurant-manager.de`.

---

5. COOLIFY — EINRICHTUNG VON A BIS Z

5.1 Was ist Coolify?

Coolify ist ein Open-Source-Tool, das deinen Server in eine einfache Deployment-Plattform verwandelt.

Analogie: Stell dir vor, du hättest einen persönlichen IT-Administrator, der rund um die Uhr auf deinem Server sitzt und:

- Deinen Code automatisch vom GitHub holt
- Die App baut und startet
- SSL-Zertifikate (HTTPS) einrichtet
- Datenbanken verwaltet
- Dir eine schöne Oberfläche gibt, statt dass du alles per Kommandozeile machen musst

Das ist Coolify. Kostenlos. Open Source.

Ohne Coolify müsstest du all das per Hand machen:

```
ssh root@server
git pull
npm install
npm run build
pm2 restart app
# SSL? nginx? Reverse Proxy? certbot? ...😵
```

Mit Coolify: Du pushst zu GitHub → alles passiert automatisch.

5.2 Coolify auf Hetzner installieren

Voraussetzungen

- Ein Hetzner VPS (CX22 oder größer) mit Ubuntu 22.04 oder 24.04
- SSH-Zugang zum Server

Installation (auf dem Server via SSH)

```
# 1. Auf den Server verbinden
ssh root@168.119.85.42

# 2. Coolify installieren (ein einziger Befehl!)
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash

# 3. Warten... (dauert 2-5 Minuten)
# Am Ende siehst du:
# "Coolify is ready! Visit http://168.119.85.42:8000"
```

Erste Einrichtung

1. Öffne im Browser: `http://DEINE-IP:8000`
2. Erstelle deinen Admin-Account (E-Mail + Passwort)
3. Folge dem Setup-Wizard:
   - Server bestätigen (Coolify erkennt den eigenen Server automatisch)
   - Fertig!

5.3 GitHub mit Coolify verbinden

Warum? Damit Coolify deinen Code von GitHub holen kann.

Schritt für Schritt

1. **In Coolify:** Gehe zu "Sources" → "Add New" → "GitHub App"
2. **Coolify generiert einen Link** — klicke darauf
3. **Auf GitHub:** Du wirst gefragt, ob Coolify Zugriff auf dein Repo bekommen darf
4. **Bestätigen** — Wähle nur das `restaurant-saas` Repo aus
5. **Zurück in Coolify:** Die Verbindung ist hergestellt

5.4 Was ist ein "Deployment"?

Deployment = Den Code auf dem Server starten, damit User ihn nutzen können.

Was passiert technisch bei einem Deployment?

```
Schritt 1: Code holen
    Coolify führt aus: git clone / git pull
    → Holt den neuesten Code von GitHub

Schritt 2: Abhängigkeiten installieren
    Coolify führt aus: npm install
    → Lädt alle benötigten Pakete herunter (React, Express, etc.)

Schritt 3: Build
    Coolify führt aus: npm run build
    → Kompiliert TypeScript zu JavaScript
    → Bündelt das Frontend zu optimierten Dateien

Schritt 4: Starten
    Coolify führt aus: npm start
    → Startet den Server
    → Die App ist live!

Schritt 5: Health Check
    Coolify prüft: Antwortet die App auf Port 3001?
    → Ja → "Deployment erfolgreich" ✅
    → Nein → "Deployment fehlgeschlagen" ❌ → alte Version bleibt live
```

5.5 App in Coolify als Projekt anlegen

Backend-Service erstellen

1. In Coolify: "Projects" → "Add New" → Name: "Restaurant SaaS"
2. "Add Resource" → "Application"
3. Source: GitHub → Wähle dein Repo → Branch: `main`
4. Build Pack: "Nixpacks" (erkennt Node.js automatisch)
5. **Wichtig — Base Directory:** `/restaurant-app/backend` (weil das Backend in einem Unterordner liegt)
6. Port: `3001`
7. Domain: `api.deine-app.de` (für das Backend)

Frontend-Service erstellen

1. Gleicher Vorgang, aber:
2. Base Directory: `/restaurant-app/frontend`

3. Build Command: `npm run build`
4. Port: Nicht nötig (statische Dateien)
5. Domain: `deine-app.de` (für das Frontend)

Datenbank erstellen

1. "Add Resource" → "Database" → "PostgreSQL"
2. Coolify erstellt automatisch eine PostgreSQL-Instanz
3. Du bekommst eine Connection-URL wie: `postgresql://postgres:generiertes-passwort@localhost:5432/coolify`
4. Diese URL trägst du als Umgebungsvariable ein (siehe nächster Abschnitt)

5.6 Environment Variables in Coolify setzen

In Coolify hat jeder Service seine eigenen Umgebungsvariablen.

Für das Backend:

Gehe zu deinem Backend-Service → "Environment Variables" → Füge hinzu:

```
DATABASE_URL = postgresql://postgres:PASSWORT@DATENBANK-HOST:5432/restaurant
JWT_SECRET = ein-langes-zufälliges-geheimnis
PORT = 3001
FRONTEND_URL = https://deine-app.de
```

Für das Frontend:

```
VITE_API_URL = https://api.deine-app.de
```

> ***Hinweis:*** *Coolify speichert die Variablen verschlüsselt. Sie sind in der Coolify-Oberfläche sichtbar, aber nirgendwo im Code.*

5.7 Automatisches Deployment (CI/CD)

CI/CD = Continuous Integration / Continuous Deployment = Dein Code wird automatisch getestet und veröffentlicht.

So funktioniert's mit Coolify + GitHub:

1. Du pushst Code zu GitHub (`git push`)
2. GitHub sendet einen **Webhook** an Coolify ("Hey, es gibt neuen Code!")
3. Coolify zieht den neuen Code
4. Coolify baut und startet die App neu
5. In 1-3 Minuten ist die neue Version live

Webhook einrichten

Das passiert meistens automatisch, wenn du GitHub über die Coolify-App verbunden hast. Falls nicht:

1. In Coolify: Kopiere die Webhook-URL (z.B. `https://coolify.deine-app.de/webhooks/...`)
2. Auf GitHub: Repo → Settings → Webhooks → Add Webhook
3. Payload URL: Die URL aus Schritt 1
4. Events: "Push events"
5. Speichern

Ab jetzt: Jeder `git push` löst automatisch ein neues Deployment aus.

5.8 SSL/HTTPS einrichten

SSL macht aus `http://` ein `https://` — das Schloss-Symbol im Browser. Ohne SSL warnt der Browser die User: "Diese Seite ist nicht sicher".

Mit Coolify ist das trivial:

1. Gehe zu deinem Service → "Settings"
2. Domain eintragen: `deine-app.de`
3. "Generate SSL Certificate" aktivieren
4. Coolify nutzt **Let's Encrypt** — komplett kostenlos!
5. Das Zertifikat wird automatisch alle 90 Tage erneuert.

Voraussetzung: Deine Domain muss bereits per DNS auf den Server zeigen (A-Record).

5.9 Datenbank in Coolify anlegen und verbinden

Schritt 1: PostgreSQL-Datenbank erstellen

1. In deinem Coolify-Projekt: "Add Resource" → "Database" → "PostgreSQL"
2. Version: 16 (oder die neueste)
3. Coolify startet einen PostgreSQL-Container
4. Du siehst die Verbindungsdaten:
   - Host: `Name-des-DB-Containers`
   - Port: `5432`
   - User: `postgres`
   - Passwort: `(automatisch generiert)`

Schritt 2: Schema importieren

```
# In Coolify: Öffne das Terminal der Datenbank
# Oder via SSH auf dem Server:

# Schema erstellen
psql $DATABASE_URL -f database/schema.sql

# Testdaten laden
psql $DATABASE_URL -f database/seed.sql
```

Schritt 3: Backend mit DB verbinden

In den Environment Variables des Backend-Services:

```
DATABASE_URL = postgresql://postgres:PASSWORT@DATENBANK-CONTAINER:5432/restaurant
```

Coolify-intern können sich Services über den Container-Namen finden (wie in einem privaten Netzwerk).

---

6. VOM CODE ZUM LIVE-PRODUKT — DER KOMPLETTE WORKFLOW

6.1 Der gesamte Workflow Schritt für Schritt

```
DEIN LAPTOP                    GITHUB                    DEIN SERVER (HETZNER)
___________                    ______                    _____________________

                                                         Coolify läuft hier
     |                           |                            |
1. Du änderst Code              |                            |
   (z.B. neue Funktion)         |                            |
     |                           |                            |
2. git add .                    |                            |
   git commit -m "..."          |                            |
     |                           |                            |
3. git push origin main         |                            |
   ————————————————————————→    |                            |
     |                     Code landed                       |
     |                     auf GitHub                        |
     |                           |                            |
     |                     4. GitHub sendet                   |
     |                        Webhook an Coolify              |
     |                           | ————————————————————→     |
     |                           |                            |
     |                           |                   5. Coolify holt Code
     |                           |                      git pull
     |                           |                            |
     |                           |                   6. Coolify baut
     |                           |                      npm install
     |                           |                      npm run build
     |                           |                            |
     |                           |                   7. Coolify startet
     |                           |                      npm start
     |                           |                            |
     |                           |                   8. Health Check OK ✅
     |                           |                            |
     |                           |                   9. Neue Version ist
     |                           |                      live unter
     |                           |                      deine-app.de
     |                           |                            |
10. Du rufst deine-app.de        |                            |
    auf und siehst die           |                            |
    neue Version!                |                            |
```

In Kurzform

1. **Ändern** → Code lokal bearbeiten
2. **Speichern** → `git add .` + `git commit -m "Beschreibung"`
3. **Hochladen** → `git push origin main`
4. **Warten** → Coolify baut automatisch (1-3 Minuten)
5. **Fertig** → Neue Version ist live

6.2 Was passiert wenn ein Build fehlschlägt?

Logs lesen in Coolify

1. Gehe zu deinem Service in Coolify
2. Klicke auf "Deployments"
3. Klicke auf das fehlgeschlagene Deployment
4. Du siehst die Build-Logs — dort steht die Fehlermeldung

Häufige Fehler und Lösungen

Fehler	Bedeutung	Lösung
`npm ERR! Missing script: "build"`	Der Build-Befehl fehlt in package.json	`"build": "tsc && vite build"` in package.json eintragen
`Cannot find module 'xyz'`	Ein Paket fehlt	`npm install xyz` ausführen und pushen
`TypeScript error TS2345`	Typ-Fehler im Code	Den Fehler im Code fixen
`ECONNREFUSED`	Datenbank nicht erreichbar	DATABASE_URL prüfen
`Port already in use`	Port ist belegt	Anderen Port wählen oder alten Prozess stoppen

Wichtig: Die alte Version bleibt live!

Wenn ein Build fehlschlägt, zeigt deine App weiterhin die letzte funktionierende Version. Deine User merken nichts davon.

6.3 Rollback — Alte Version wiederherstellen

Rollback = Zur vorherigen funktionierenden Version zurückkehren.

In Coolify:

1. Gehe zu "Deployments"
2. Finde das letzte erfolgreiche Deployment
3. Klicke "Redeploy"
4. Fertig — die alte, funktionierende Version ist wieder live

Per Git:

```
# Letzte 5 Commits anzeigen
git log --oneline -5

# Ergebnis z.B.:
# a1b2c3d Neues Feature (kaputt)
# e4f5g6h Login-Seite hinzugefügt (funktioniert)
# ...

# Zum funktionierenden Stand zurück
git revert a1b2c3d
git push origin main
# → Coolify deployed automatisch die reparierte Version
```

`git revert` macht die Änderungen des angegebenen Commits rückgängig, OHNE die Geschichte zu löschen. Das ist sicherer als `git reset`.

7. SICHERHEIT — WAS DU WISSEN MUSST

7.1 API Keys und Geheimnisse schützen

Was sind API Keys?

Analogie: Ein API-Key ist wie ein Schlüssel zu einem bestimmten Dienst. Wer den Schlüssel hat, kann den Dienst nutzen — auf deine Kosten.

Beispiele:

- **DATABASE_URL** — Zugang zur Datenbank (= alle deine Daten)
- **JWT_SECRET** — Damit werden Login-Tokens signiert
- **STRIPE_SECRET_KEY** — Zugang zu deiner Zahlungsabwicklung

Regeln

1. **Niemals in den Code schreiben** — Immer in `.env` oder Coolify-Umgebungsvariablen
2. **Niemals auf GitHub pushen** — `.env` MUSS in `.gitignore` stehen
3. **Verschiedene Keys für Entwicklung und Produktion** — Was lokal funktioniert, ist ein anderer Key als auf dem Live-Server
4. **Keys regelmäßig rotieren** — Alle 3-6 Monate neue Keys generieren

---

7.2 Server-Sicherheit

Die Basics

Maßnahme	Was es macht	Wie
SSH-Key statt Passwort	Verhindert Brute-Force-Angriffe	`ssh-keygen` + Passwort-Login deaktivieren
Firewall (ufw)	Blockiert unerwünschte Verbindungen	Nur Ports 22, 80, 443 öffnen
Updates	Schließt bekannte Sicherheitslücken	`apt update && apt upgrade` regelmäßig
Fail2Ban	Sperrt IPs nach zu vielen Fehlversuchen	`apt install fail2ban`

Firewall einrichten

```
# Auf dem Server:
ufw allow 22    # SSH (damit du dich verbinden kannst)
ufw allow 80    # HTTP (wird zu HTTPS weitergeleitet)
ufw allow 443   # HTTPS (verschlüsselte Verbindungen)
ufw enable      # Firewall aktivieren
```

Was die Ports bedeuten

Port	Dienst	Warum offen?
22	SSH	Damit du dich per Terminal verbinden kannst
80	HTTP	Damit Besucher die Seite aufrufen können (leitet zu 443 weiter)
443	HTTPS	Verschlüsselte Verbindungen (die eigentliche App)
8000	Coolify UI	Nur während der Einrichtung offen lassen. Danach über Domain absichern.

Alle anderen Ports sind geschlossen. Die Datenbank (Port 5432) ist nur intern erreichbar — von außen kommt niemand ran.

7.3 Backups

Was sind Backups?

Analogie: Wie eine Fotokopie aller wichtigen Dokumente deines Restaurants, die du in einem anderen Gebäude aufbewahrst. Wenn das Restaurant abbrennt, hast du noch die Kopien.

Was musst du sichern?

Was	Warum	Wie oft
Datenbank	Alle Kundendaten, Bestellungen, Reservierungen	Täglich
Umgebungsvariablen	Deine .env-Konfiguration	Bei jeder Änderung
Code	Dein gesamter Code	Automatisch (liegt auf GitHub)

Datenbank-Backup einrichten

In Coolify:

1. Gehe zur Datenbank-Ressource
2. "Backups" → Zeitplan einstellen (z.B. täglich um 3:00 Uhr)
3. Speicherort: Entweder lokal auf dem Server oder S3-kompatibel (z.B. Hetzner Object Storage)

Manuell (im Notfall):

```
# Backup erstellen
pg_dump $DATABASE_URL > backup_2026-04-05.sql

# Backup wiederherstellen
psql $DATABASE_URL < backup_2026-04-05.sql
```

Die 3-2-1 Regel für Backups

- **3** Kopien deiner Daten
- **2** verschiedene Speichermedien (Server + externer Speicher)
- **1** Kopie an einem anderen Ort (z.B. Hetzner Object Storage)

---

8. VERÖFFENTLICHUNG & LAUNCH

8.1 Launch-Checkliste

Technik

- [ ] Server läuft (Hetzner VPS)
- [ ] Coolify ist installiert und konfiguriert
- [ ] Domain gekauft und DNS eingerichtet
- [ ] SSL/HTTPS aktiv (Let's Encrypt)
- [ ] Backend deployed und erreichbar
- [ ] Frontend deployed und erreichbar
- [ ] Datenbank läuft mit aktuellem Schema
- [ ] Automatisches Deployment funktioniert (Push → Live)
- [ ] Backups eingerichtet (täglich)
- [ ] Firewall konfiguriert
- [ ] Alle Umgebungsvariablen gesetzt

Sicherheit

- [ ] `.env` ist in `.gitignore`
- [ ] Passwörter sind mit bcrypt gehasht
- [ ] JWT-Token laufen nach angemessener Zeit ab
- [ ] Multi-Tenant: `restaurant_id` wird IMMER aus dem Token gelesen
- [ ] API-Preise kommen aus der DB, nie vom Client
- [ ] Passwort-Hashes werden NIE in API-Responses zurückgegeben

DSGVO

- [ ] Server steht in Deutschland/EU
- [ ] Datenschutzerklärung auf der Website
- [ ] Impressum auf der Website
- [ ] Cookie-Banner (falls Cookies über die Session hinaus genutzt werden)

- [ ] Personenbezogene Daten können gelöscht werden (Recht auf Löschung)
- [ ] Telefonnummern in Reservierungen werden nach 30 Tagen gelöscht
- [ ] Verarbeitungsverzeichnis geführt (`project/dsgvo-datenkatalog.md`)
- [ ] AV-Vertrag (Auftragsverarbeitungsvertrag) mit Hetzner abschließen

Geschäftlich

- [ ] Zahlungsanbieter eingerichtet (Stripe)
- [ ] AGB erstellt
- [ ] Widerrufsbelehrung (bei Abo-Modell)
- [ ] Support-Kanal definiert (E-Mail, WhatsApp?)
- [ ] Monitoring eingerichtet (weißt du wenn die App down ist?)

8.2 Domain kaufen

Empfohlene Anbieter für DACH

Anbieter	.de Domain	Vorteile
Hetzner	~6€/Jahr	Alles bei einem Anbieter (Server + Domain)
Namecheap	~8€/Jahr	Gute Benutzeroberfläche, günstige Preise
INWX	~5€/Jahr	Deutscher Anbieter, viele TLDs

Worauf achten

- **Kurzname** — Einfach zu merken und zu tippen
- **.de Domain** — Für den DACH-Markt am vertrauenswürdigsten
- **Whois-Schutz** — Deine persönlichen Daten werden nicht öffentlich angezeigt (bei .de Domains automatisch durch die DENIC geschützt)
- **Auto-Renewal aktivieren** — Damit die Domain nicht versehentlich ausläuft

---

8.3 Zahlungen mit Stripe

Warum Stripe?

- Funktioniert in Deutschland, Österreich und der Schweiz
- Unterstützt Abo-Zahlungen (monatlich/jährlich)
- Hat eine hervorragende Dokumentation
- Kann SEPA-Lastschrift (wichtig für DACH!)

Grundkonzept

```
Restaurant-Betreiber ──→ Wählt Abo-Plan ──→ Stripe Checkout
                                                  │
                                                  ▼
                                             Zahlung erfolgt
                                                  │
                                                  ▼
                                        Stripe Webhook an dein Backend
                                                  │
                                                  ▼
                                        Backend setzt abo_status = 'active'
```

Kosten

Zahlungsmethode	Gebühr pro Transaktion
Kreditkarte (EU)	1,5% + 0,25€
SEPA-Lastschrift	0,35€
Giropay	1,5% + 0,25€

8.4 DSGVO für deine SaaS

Die 5 wichtigsten Punkte

1. **Server-Standort** — MUSS in der EU sein (Hetzner = Deutschland ✅)
2. **Datenschutzerklärung** — Muss auf der Website sein. Erklärt welche Daten du sammelst und warum.
3. **AV-Vertrag** — Du verarbeitest Daten im Auftrag der Restaurants. Brauchst einen "Auftragsverarbeitungsvertrag" mit jedem Restaurant-Kunden UND mit Hetzner.
4. **Recht auf Löschung** — Jeder kann verlangen, dass seine Daten gelöscht werden. Technisch muss das möglich sein.
5. **Datenminimierung** — Nur die Daten speichern, die du wirklich brauchst.

Was in deiner App bereits DSGVO-relevant ist

Datenfeld	Tabelle	Personenbezogen?	Löschfrist
telefon	reservierungen	Ja	30 Tage nach Datum
email	mitarbeiter	Ja	Bei Konto-Löschung
gast_name	reservierungen	Ja	30 Tage nach Datum
passwort_hash	mitarbeiter	Ja (Pseudonym)	Bei Konto-Löschung

8.5 Monitoring

Was ist Monitoring?

Du willst sofort wissen, wenn deine App nicht erreichbar ist — BEVOR deine Kunden sich bei dir melden.

Einfachste Lösung: UptimeRobot

1. Gehe auf uptimerobot.com (kostenloser Plan reicht)
2. Erstelle einen Monitor:
   - URL: `https://deine-app.de`
   - Intervall: 5 Minuten
   - Alarmierung: E-Mail und/oder SMS
3. Wenn die App down ist → Du bekommst sofort eine Nachricht

Was du überwachen solltest

Was	URL/Endpoint	Prüft
Frontend	`https://deine-app.de`	Ist die Website erreichbar?
Backend-API	`https://api.deine-app.de/api/health`	Antwortet das Backend?
Datenbank	(über Backend Health-Check)	Ist die DB verbunden?

---

9. KOSTENZUSAMMENFASSUNG

9.1 Monatliche / Jährliche Kosten

Posten	Anbieter	Kosten	Anmerkung
Server (VPS)	Hetzner CX22	**~4,50€/Monat**	2 vCPU, 4 GB RAM, 40 GB SSD
Domain (.de)	Hetzner / INWX	**~6€/Jahr**	= 0,50€/Monat
SSL-Zertifikat	Let's Encrypt	**0€**	Kostenlos, automatisch via Coolify
GitHub	GitHub Free	**0€**	Private Repos sind kostenlos
Coolify	Self-hosted	**0€**	Open Source, läuft auf deinem Server
Monitoring	UptimeRobot Free	**0€**	50 Monitore kostenlos
E-Mail	(Optional)	1-5€/Monat	Für Transaktions-E-Mails (z.B. Mailgun)
Backups	Hetzner Backup	**~1,20€/Monat**	20% des Server-Preises

Stripe-Gebühren (nur bei Umsatz)

Zahlungsart	Gebühr
Kreditkarte (EU)	1,5% + 0,25€ pro Zahlung
SEPA-Lastschrift	0,35€ pro Zahlung

9.2 Minimum um live zu gehen

Posten	Kosten
Hetzner VPS	4,50€/Monat
Domain	6€/Jahr
Gesamt (1. Jahr)	**~60€/Jahr = ~5€/Monat**

Alles andere (SSL, GitHub, Coolify, Monitoring) ist kostenlos.

Das ist dein gesamter Einstieg: ~5€/Monat. Zum Vergleich: Ein einzelner Kaffee kostet mehr.

---

10. WIE ALLES ZUSAMMENHÄNGT — DIE GESAMTÜBERSICHT

```
                    ╔═══════════════════════╗
                    ╚═══════════════════════╝
                    ╚═══════════════════════╝
                                 │
                                 │ restaurant-manager.de
                                 │
                                 ▼
                    ┌─────────────────────┐
                    │       DNS-Server    │
                    │ "restaurant-manager.de
                    │  → 168.119.85.42"   │
                    └─────────────────────┘
                                 │
                                 │
                                 ▼
┌──────────────────────────────────────────────────────────┐
│                DEIN HETZNER VPS SERVER                   │
│                (168.119.85.42)                           │
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │                   COOLIFY                        │    │
│  │                                                  │    │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────┐  │    │
│  │  │   FRONTEND   │  │   BACKEND    │  │ POSTGRESQL │  │    │
│  │  │              │  │              │  │          │  │    │
│  │  │ React +      │  │ Node.js +    │  │ Alle     │  │    │
│  │  │ TypeScript   │──│ Express +    │──│ Daten    │  │    │
│  │  │ + Tailwind   │  │ Socket.io    │  │          │  │    │
│  │  │              │  │              │  │ restaurants │  │    │
│  │  │deine-app.de  │  │ api.deine-   │  │ tische   │  │    │
│  │  │              │  │ app.de       │  │ gerichte │  │    │
│  │  │ Port: 443    │  │ Port: 3001   │  │ bestellungen│  │    │
│  │  │ (HTTPS)      │  │              │  │ ...      │  │    │
│  │  └──────────────┘  └──────────────┘  └──────────┘  │    │
│  │                          │                       │    │
│  │                   Socket.io                      │    │
│  │                   (Live-Updates)                 │    │
│  │                          │                       │    │
│  └──────────────────────────┴───────────────────────┘    │
│                             │                            │
└─────────────────────────────┴────────────────────────────┘
                             │
                             │
                             │ Webhook: "Neuer Code!"
                             │
                             │
        ┌────────────────────┴────────────────────┐
        │                    │                    │
        ▼                    │                    ▼
 ┌──────────┐                │             ┌──────────┐
 │  GITHUB  │                │             │ BENUTZER │
 │          │                │             │          │
 │restaurant-│──── Push ────→│             │Restaurant:│
 │saas Repo │  triggert      │             │├ Admin   │
 │          │  Deployment    │             │├ Kellner │
 │Code +    │                │             │├ Küche   │
 │Verlauf   │                │             │└ Gast (QR)│
```

```
└──────────┘                                  └──────────────────┘
       │
       │
       │ git push
       │
       │
┌──────┴──────┐
│ DEIN LAPTOP │
│             │
│ VS Code +   │
│ Claude Code │
│             │
│ Hier        │
│ entwickelst │
│ du          │
└─────────────┘
```

Der Datenfluss in einem Bild

```
                    ┌──────────────────────────────────┐
                    │         DER LEBENSZYKLUS         │
                    │       EINER BESTELLUNG           │
                    └──────────────────────────────────┘

  GAST                                              KÜCHE
 (Handy)                                           (Tablet)
   │                                                  ▲
   │ 1. Scannt QR-Code                                │
   │    am Tisch                                      │ 6. Bestellung
   ▼                                                  │    erscheint live
┌──────────┐                                          │    (Socket.io)
│ FRONTEND │                                          │
│ (React)  │                                       ┌──┴──────┐
│          │  2. POST /api/bestellungen            │ FRONTEND │
│ Bestell- │──────────────────────────────────────→│ (React)  │
│ formular │                                       │          │
└──────────┘                                       │ Küchen-  │
                                                   │ Ansicht  │
                    ┌────────────────┐             └──────────┘
                    │    BACKEND     │                  ▲
                    │   (Express)    │                  │
                    │                │                  │
                    │ 3. Prüft Daten │                  │
                    │ 4. Holt Preise │          5. Socket.io
                    │    aus DB      │             Push
                    │                │                  │
                    └────────┬───────┘                  │
                             │                          │
                             │                          │
                  5. INSERT│INTO                         │
                             │                          │
                    ┌────────▼───────┐                  │
                    │  POSTGRESQL    │                  │
                    │                │                  │
                    │ bestellungen   │──────────────────┘
                    │ bestellpos.    │   Trigger/Notify
                    └────────────────┘
```

Glossar — Die wichtigsten Begriffe

Begriff	Erklärung
API	Schnittstelle, über die Programme miteinander kommunizieren
Backend	Der unsichtbare Teil der App (Server, Logik, Datenbank)
Branch	Ein paralleler Entwicklungszweig in Git
Build	Den Code in eine lauffähige Version umwandeln
CI/CD	Automatisches Testen und Veröffentlichen von Code
CLI	Command Line Interface — Textbasierte Eingabe im Terminal
Clone	Ein Repo von GitHub auf den eigenen Computer kopieren
Commit	Ein gespeicherter Schnappschuss des Code-Zustands
Container	Eine isolierte Umgebung, in der eine App läuft (wie eine Mini-VM)
CORS	Sicherheitsregel, die festlegt welche Websites auf deine API zugreifen dürfen
CRUD	Create, Read, Update, Delete — die 4 Grundoperationen auf Daten
Deployment	Den Code auf einen Server bringen und starten
DNS	Das "Telefonbuch des Internets" — übersetzt Domains in IP-Adressen
Domain	Der Name deiner Website (z.B. deine-app.de)
DSGVO	Datenschutz-Grundverordnung — EU-Gesetz für Datenschutz
Endpoint	Eine bestimmte URL, an die das Frontend Anfragen schickt
ENV	Environment Variable — Konfigurationswert der nicht im Code steht
Firewall	Software die unerwünschte Netzwerkverbindungen blockiert
Foreign Key	Verweis von einer Tabelle auf eine andere (Beziehung)
Frontend	Der sichtbare Teil der App (UI, Design, Interaktion)
Git	Versionsverwaltung — zeichnet jede Code-Änderung auf
GitHub	Online-Plattform für Git-Repositories
Hash	Einweg-Verschlüsselung (z.B. für Passwörter)
HTTP/HTTPS	Protokoll für Webkommunikation (S = verschlüsselt)
JSON	Textformat für strukturierte Daten

JWT	JSON Web Token — digitaler "Ausweis" nach dem Login
Middleware	Code der zwischen Anfrage und Antwort läuft (z.B. Auth-Prüfung)
Multi-Tenant	Mehrere Kunden teilen sich eine App, sehen aber nur eigene Daten
ORM	Object-Relational Mapping — Datenbank-Abfragen als Code statt SQL
Port	Eine "Tür" auf dem Server (z.B. 443 = HTTPS, 22 = SSH)
Primary Key	Eindeutiger Identifikator einer Zeile in einer Tabelle (z.B. `id`)
Pull	Änderungen von GitHub auf den eigenen Computer holen
Push	Lokale Änderungen zu GitHub hochladen
Query	Eine Datenbankabfrage (z.B. `SELECT * FROM tische`)
Repo	Repository — ein Git-Projekt mit Code und Verlauf
REST	Regelwerk für APIs (GET, POST, PUT, DELETE)
Rollback	Zur vorherigen funktionierenden Version zurückkehren
Schema	Der Bauplan einer Datenbank (welche Tabellen, welche Felder)
SQL	Structured Query Language — Sprache für Datenbank-Abfragen
SSH	Sichere Verbindung zum Server per Terminal
SSL	Verschlüsselung für HTTPS (das Schloss im Browser)
Tenant	Ein "Mieter" in einer Multi-Tenant-App (= ein Restaurant)
Token	Ein digitaler Schlüssel (z.B. JWT für Login)
UUID	Universally Unique Identifier — eine garantiert einzigartige ID
VPS	Virtual Private Server — dein eigener virtueller Server
Webhook	Automatische Benachrichtigung von einem Service an einen anderen
WebSocket	Permanente Verbindung für Echtzeit-Kommunikation (Socket.io nutzt das)

> ***Nächster Schritt:*** *Server bei Hetzner bestellen (CX22, Ubuntu 22.04, Standort Falkenstein) und Coolify installieren. Dann können wir deine App live deployen.*